Assignment 3

Thread Scaling in Shared-Memory Matrix Multiplication

Abel Castro Villar
Data Science and Engineering
Universidad de Las Palmas de Gran Canaria (ULPGC)
Big Data | Academic year 2026-2027
github.com/abelcasv/Assignment03_Abel

Abstract. This study asks how OpenMP thread count affects the runtime and memory use of dense square matrix multiplication, and when parallel execution helps on one hybrid CPU. A C11 loop-interchanged kernel is compared with its sequential counterpart on identical binary64 inputs for six dimensions and eight requested team sizes. Each configuration has one warm-up and five measured repetitions; results are reported as medians and interquartile ranges. For n=2000, median time falls from 15.259 s sequentially to 1.377 s with 12 OpenMP threads (11.08x speedup; 92.4% efficiency). The OpenMP one-thread run is slower than the sequential baseline at this size. Scaling is not monotonic at every size, and a secondary blocked algorithm has a different thread-count profile. The evidence supports parallel execution for the larger tested matrices on this host, but does not identify individual core placement or establish a universal optimum.

1. Introduction

Dense matrix multiplication is useful for studying shared-memory parallelism because its arithmetic work grows as O(n^3), while three dense n x n binary64 arrays require 24n^2 bytes. This report follows Assignment 3's requirement to compare equivalent sequential and parallel implementations, vary thread count, validate correctness, and report time and memory with enough detail to reproduce the measurements [1].

Question and hypotheses

The research question is: how does OpenMP team size affect runtime and memory use for dense matrix multiplication, and at which tested matrix sizes does the parallel version outperform its sequential counterpart on this host? Before measurement, we expected small workloads to be sensitive to parallel-region overhead, larger workloads to benefit from multiple threads, and performance to stop improving once thread count exceeded the available execution contexts. These are predictions; the results test them and do not assume them to be true.

2. Methods

Implementations and inputs

The primary sequential kernel and its OpenMP counterpart use row-major double arrays and the same i-k-j loop order. The OpenMP version statically distributes the outer i loop; each iteration writes a separate row of C, avoiding shared writes to the same output elements. A second blocked implementation uses 32 x 32 tiles, a common locality technique for matrix multiplication [2], and is compared only with its own sequential version. It is a secondary algorithm comparison, not a substitute baseline. Matrix input is shared by every method at a given size.

The generator uses Python random.Random(42), draws each element using random() in [0,1), and writes six decimal places to CSV. Rounding can produce stored values of 1.000000; the observed CSV range is [0,1]. Values are read as IEEE 754 binary64. Dimensions are n in {250, 500, 750, 1000, 1500, 2000}. Python documents this generator as using Mersenne Twister; reproducibility is tied to the included generator and saved stream, not to matching random sequences in other languages [3]. The script writes data/SHA256SUMS.txt for the 12 dense inputs and five correctness fixtures. The dense files were regenerated and verified to have matching SHA-256 hashes.

Host and execution configuration

Item	Recorded configuration
CPU	Intel Core i5-1335U; 10 physical cores (2 P-cores + 8 E-cores); 12 logical processors [4]
Memory / OS	15.63 GiB physical RAM; Windows build 26300
Compiler / flags	MSYS2 UCRT64 GCC 16.1.0; -Wall -Wextra -O3 -std=c11 -fopenmp -march=native
OpenMP / analysis	API 5.2 (_OPENMP=202111) [6]; Python Python 3.12.14 (64-bit, MSVC 1944); NumPy 2.5.4; pandas 3.0.1; Matplotlib 3.11.2

Requested team sizes were p in {1, 2, 4, 8, 10, 12, 16, 24}; every recorded actual team size matched its request. Dynamic adjustment was disabled, OMP_PROC_BIND=false and OMP_PLACES was unset. No explicit affinity was used, so worker placement on P- or E-cores is unknown. Requests above 12 are labelled oversubscribed relative to the host's logical processor count.

Timing, memory and correctness

Inputs are loaded and arrays allocated outside timing. A monotonic Windows QueryPerformanceCounter timer surrounds one kernel call. It includes zero-initialisation of C and, for OpenMP, team and loop overhead; it excludes file input, allocation, and a checksum over every output element. The checksum is consumed through a volatile sink. One warm-up is excluded; five measured repetitions are retained per configuration. The reported centre is the median; quartiles are NumPy's 25th and 75th percentiles, and IQR is Q3-Q1.

Known-answer, identity and zero 4 x 4 fixtures are checked, followed by comparison of every kernel and tested OpenMP size against a trusted n=250 reference. A result passes when the absolute difference is at most 1e-4 + 1e-4 times the reference magnitude. The complete benchmark repeated these checks successfully. A per-repetition 120 s stop threshold and a three-array structural-storage budget of 2048 MiB were set before the full run. The time threshold is checked after a kernel call returns; if exceeded, that sample would be retained and the remaining repetitions skipped. The largest size tested was n=2000; larger sizes were not attempted. No call exceeded the time threshold or the memory budget. The memory budget estimates matrix-array storage, not total process memory.

Windows process peak working set is recorded through GetProcessMemoryInfo [5]. Because all configurations run in one process and this is a high-water mark, CSV values are not independent per-configuration peaks. The report gives the overall observed process peak and per-size structural estimates separately.

3. Results

The validated dataset contains 540 observations: 108 configurations with five repetitions each. It covers six sizes, two algorithms (loop-interchanged and blocked), and eight OpenMP team sizes, with a matched sequential baseline per algorithm and size. The analysis checks schema, full grid, repetitions, duplicates, finite positive times, team sizes and oversubscription labels before generating summaries.

Primary thread-scaling comparison

n	Sequential ikj median [Q1, Q3] (s)	OpenMP p=12 median [Q1, Q3] (s)	S(12)	E(12)
500	0.0595 [0.0590, 0.0641]	0.0158 [0.0157, 0.0182]	3.77x	31.5%
1000	1.2341 [1.0738, 1.6746]	0.2079 [0.2069, 0.2115]	5.93x	49.5%
1500	3.2699 [3.1973, 3.2991]	0.8397 [0.8164, 0.9137]	3.89x	32.5%
2000	15.2589 [14.2205, 15.9683]	1.3768 [1.3105, 1.3896]	11.08x	92.4%

At n=2000, the median falls from 15.259 s sequentially to 1.377 s with 12 OpenMP threads, a matched speedup of 11.08x. At n=500 the speedup is 3.77x; at n=1500 it is 3.89x. Results do not increase smoothly with size: the sequential n=1000 sample has a broad IQR, and the 12-thread speedup at n=1500 is lower than at n=1000 or n=2000. Results are therefore compared per size rather than summarized as one scaling trend.

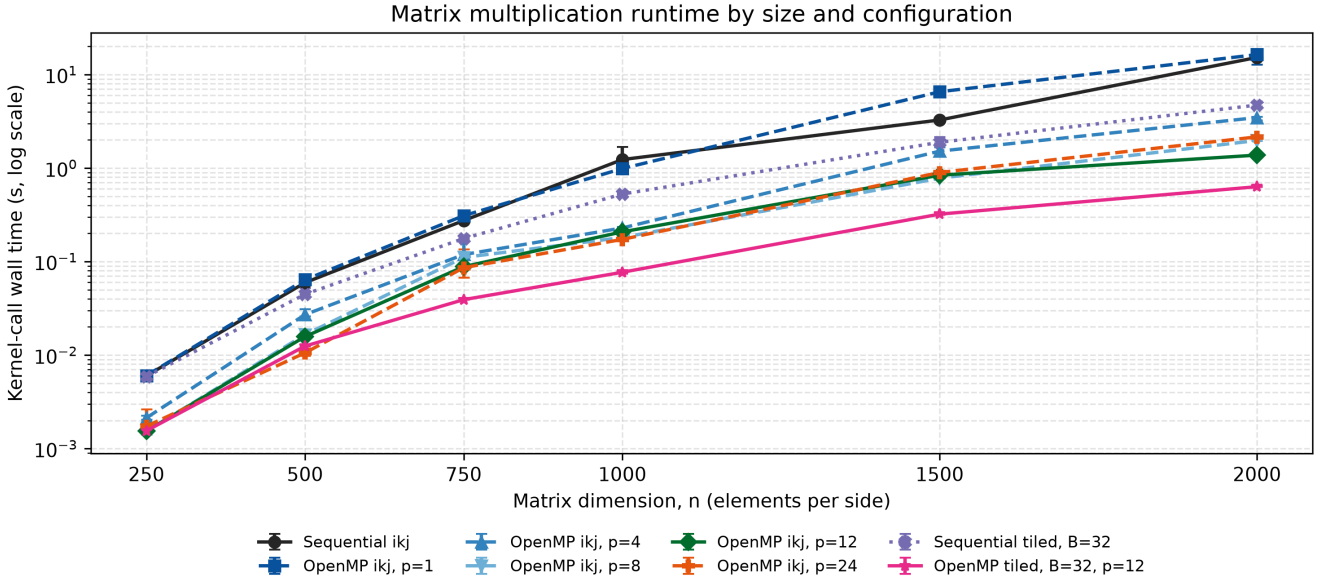


Figure 1. Runtime medians and IQR by matrix size for selected sequential and OpenMP configurations. The vertical axis is logarithmic.

OpenMP one-thread overhead

n	OpenMP p=1 difference from sequential median
250	+0.9%
500	+7.5%
750	+12.8%
1000	-19.8%
1500	+100.5%
2000	+7.3%

The direction of the one-thread difference varies. It is 0.9% slower at n=250, 7.5% slower at n=500, 12.8% slower at n=750, and 100.5% slower at n=1500; it is 19.8% and 7.3% faster in the measured n=1000 and n=2000 samples. This is not a fixed runtime penalty; variability and system state affect the observed comparison.

Thread scaling at n=2000

p	OpenMP median [Q1, Q3] (s)	S(p)	E(p)
1	16.3745 [12.8002, 17.4957]	0.93x	93.2%
2	5.8807 [3.8517, 7.6126]	2.59x	129.7%
4	3.4712 [3.2525, 3.5302]	4.40x	109.9%
8	1.9744 [1.9540, 1.9768]	7.73x	96.6%
10	1.5443 [1.4599, 1.5939]	9.88x	98.8%
12	1.3768 [1.3105, 1.3896]	11.08x	92.4%
16	2.3040 [1.6423, 2.5708]	6.62x	41.4%
24	2.1583 [2.0985, 2.2466]	7.07x	29.5%

At n=2000, speedup rises from 0.93x at one OpenMP thread to 11.08x at p=12, then falls to 6.62x at p=16 and 7.07x at p=24. The 12-thread configuration has the best observed primary-kernel median. The oversubscribed results are slower than p=12, but p=24 is slightly faster than p=16; the data do not show a strictly monotonic oversubscription penalty.

The p=2 and p=4 medians exceed ideal linear efficiency. Team sizes were verified; inputs and timing boundaries were matched; all samples were retained; and correctness passed. The p=2 IQR is wide, and no hardware counters, frequency logs or affinity were recorded. Cache, frequency or scheduling effects are plausible explanations but are not demonstrated. Superlinear values are observations that warrant further controlled investigation, not proof of a specific cause.

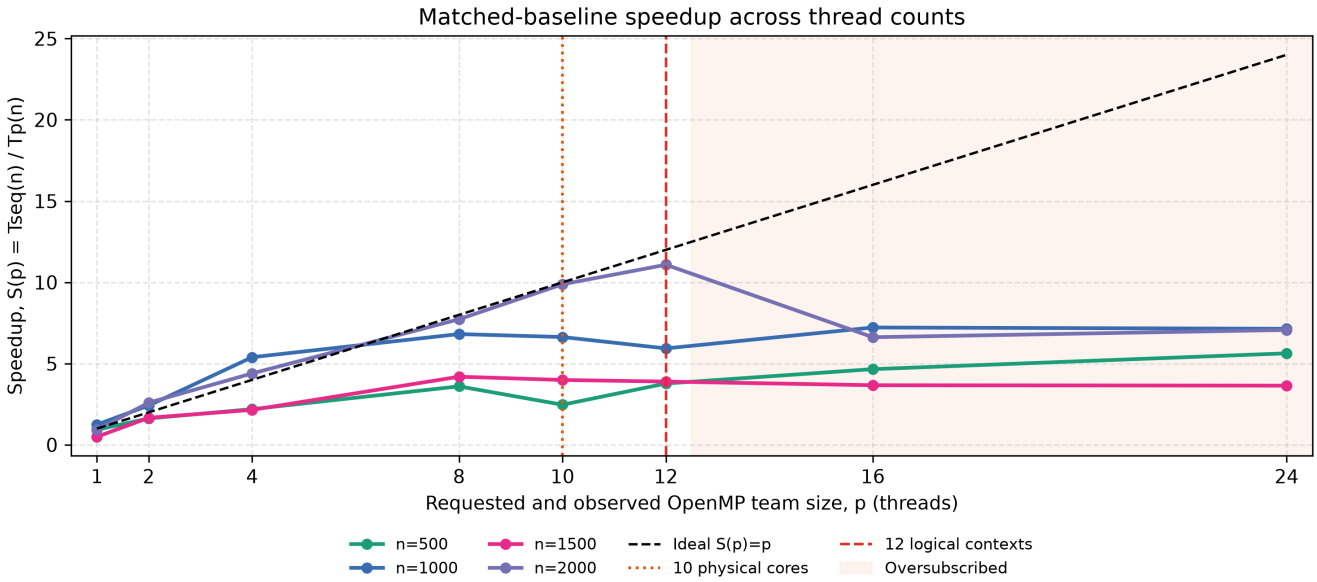


Figure 2. Matched-baseline speedup for the primary kernel. Vertical markers indicate hardware counts, not worker placement.

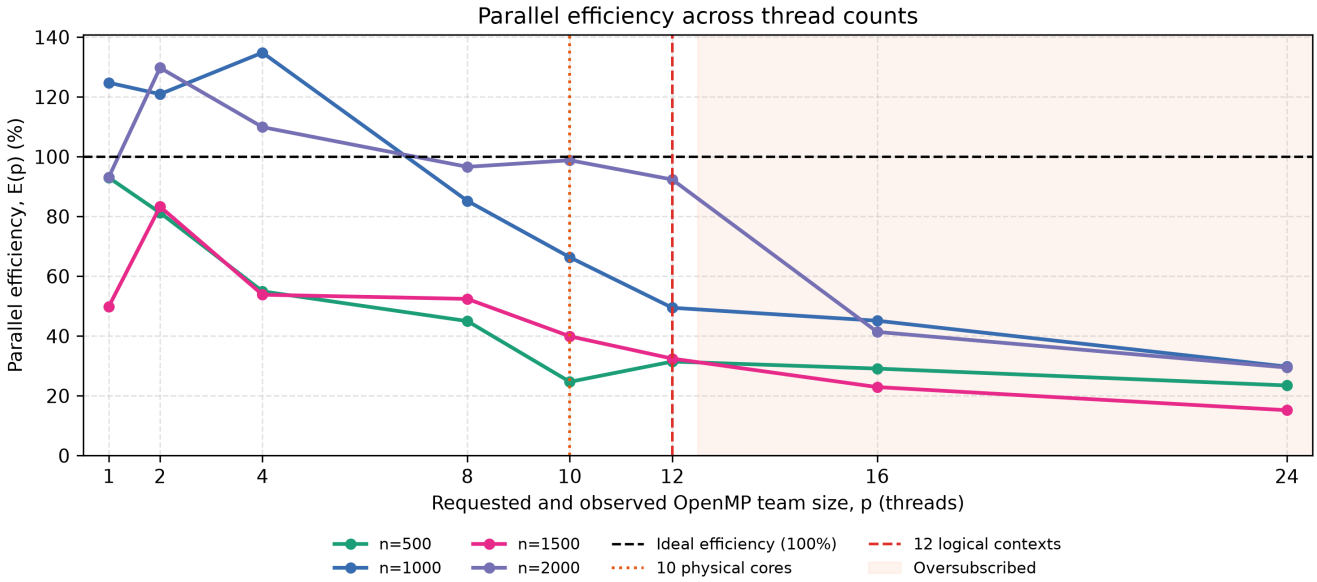


Figure 3. Parallel efficiency E(p)=S(p)/p. Values over 100% remain visible; shading marks requests above 12 logical processors.

Secondary blocked-kernel comparison and memory

Configuration	Median [Q1, Q3] (s)	Matched speedup	Efficiency
Sequential tiled B=32	4.7202 [4.5675, 5.1463]	1.00x	100.0%
OpenMP tiled p=8	0.6026 [0.5980, 0.6144]	7.83x	97.9%
OpenMP tiled p=10	0.6250 [0.6184, 0.6355]	7.55x	75.5%
OpenMP tiled p=12	0.6325 [0.6267, 0.6372]	7.46x	62.2%
OpenMP tiled p=16	0.7015 [0.6783, 0.7086]	6.73x	42.1%
OpenMP tiled p=24	0.5826 [0.5790, 0.5833]	8.10x	33.8%

At n=2000, sequential tiled multiplication has a median of 4.7202 s, compared with 15.2589 s for sequential ikj. This is an algorithm difference, not a pure thread-count effect. The blocked OpenMP kernel reaches 0.6026 s at p=8 and 0.5826 s at p=24. The latter is the fastest observed blocked median despite oversubscription; p=16 is slower. The small p=8/p=24 difference should be interpreted cautiously and does not support a general claim that oversubscription helps.

n	Three-array structural estimate (MiB)
250	1.43
500	5.72
750	12.87
1000	22.89
1500	51.50
2000	91.55
Overall process peak working set	97.36 MiB

At n=2000, three arrays require an estimated 91.55 MiB; the observed process high-water working set across the full run is 97.36 MiB. The process metric also includes runtime and other resident pages, and is cumulative; it cannot be treated as an independent peak for every row. No size exceeded the stated structural budget.

4. Discussion and limitations

The required matched comparison answers the research question within the tested scope: at n=2000, 12-thread OpenMP is substantially faster than sequential ikj, while OpenMP with one thread is not faster. Smaller-size results vary. The smallest kernels have short durations, and the n=1000 sequential baseline has a broad IQR, so a thread count should not be selected from one measurement alone.

Thread scaling is not monotonic across every configuration. At n=2000, the primary kernel is fastest at p=12, with slower medians at 16 and 24; in the blocked extension the fastest observed median is at 24. These results do not isolate scheduling, thermal state, CPU frequency, or P/E-core placement. A controlled follow-up with recorded affinity and hardware performance counters is needed for causal explanation.

The study uses five repetitions and one full run on a single Windows host. Process peak working set is not an isolated per-configuration metric. Inputs are rounded to six decimal places; outputs use explicit absolute and relative tolerances. The tested range ends at n=2000; no claim is made for larger matrices or other processors. SIMD is optional under the assignment and is excluded from the final core results; no SIMD conclusion is drawn.

5. Conclusion

The main comparison uses identical inputs, data representation and loop order for sequential and OpenMP kernels, validates correctness, and records five repetitions for each of 108 configurations. At n=2000, 12 OpenMP threads reduce the median primary-kernel time from 15.259 s to 1.377 s (11.08x speedup; 92.4% efficiency). The one-thread and oversubscribed results show that overhead and scaling depend on the configuration. These conclusions apply to the measured implementation and host only. Raw data, the deterministic generator, analysis script, environment records, exact commands, report source and this reproducible PDF are included in the repository.

AI assistance declaration

Gemini was used to draft the initial CLAUDE.md project guidance and prompt plan. Claude was used to carry out implementation and documentation work through those prompts. OpenAI Codex was used for an independent audit and subsequent corrections to the benchmark, analysis, reproducibility materials and report. AI-generated material is not evidence; the student remains responsible for understanding and defending the code, measurements, interpretation and references.

References

[1] Big Data Course Team, *BIG DATA | Individual Assignments | Revised proposal 2026-2027*, Assignment 3 and common protocol, course document, 2026.

[2] K. Goto and R. A. van de Geijn, 'Anatomy of high-performance matrix multiplication,' *ACM Transactions on Mathematical Software*, vol. 34, no. 3, Article 12, 2008. doi:10.1145/1356052.1356053.

[3] Python Software Foundation, 'random - Generate pseudo-random numbers,' Python 3 documentation. docs.python.org/3/library/random.html. Accessed Nov. 1, 2026.

[4] Intel Corporation, 'Intel Core i5-1335U Processor Specifications.' Intel product specification. Accessed Nov. 1, 2026.

[5] Microsoft, 'Process Memory Usage Information,' Windows API documentation. Microsoft Learn. Accessed Nov. 1, 2026.

[6] OpenMP Architecture Review Board, *OpenMP Application Programming Interface Version 5.2*, Nov. 2021. OpenMP specification.